

Algorithmics	Student information	Date	Number of session
	UO: 307423	30/04	7
	Surname: Miranda Martín	 Escuela de Ingeniería Informática Universidad de Oviedo	
	Name: Mariana Patricia		



Activity 1. [Bin Packing with Branch and Bound]

PART A: Explain what the proposed branching heuristic consists of.

The purposed heuristic function estimates the total number of bins required by adding the number of bins currently in use and an optimistic guess of the bins needed for the remaining objects.

$$h(n) = \text{bins already used} + \left\lceil \frac{\text{remaining size}}{\text{capacity}} \right\rceil$$

In this way, the algorithm prioritizes the nodes (paths) that have better chances of being the optimal solution.

PART B: A class BinPackingBaB.java that after being executed in the same way as in the previous session, offers a solution to the problem, ideally improving the times that had been obtained.

You can see BinPackingBaB.java along with Node.java implementation in GitHub.

The tests are working as expected, as you can see below.

```

BinPackingBaBTests [Runner: JUnit 5] (0,071 s)
  test00 (0,000 s)
  test01 (0,003 s)
  test02 (0,008 s)
  test03 (0,003 s)
  test04 (0,006 s)
  test05 (0,006 s)
  test06 (0,003 s)
  test07 (0,005 s)
  test08 (0,012 s)
  test09 (0,022 s)

```

Algorithmics	Student information	Date	Number of session
	UO: 307423	30/04	7
	Surname: Miranda Martín		
	Name: Mariana Patricia		

PART C: A class `BinPackingBaBTimes.java` that will run all the test files to calculate the time needed to solve the problem. If the program takes too long for any of them, it can be stopped for that specific case. Case 3 and, especially, case 4 are challenging.

You can see `BinPackingBaBTimes.java` implementation in [GitHub](#).

PART D: Finally, fill in a table showing the times obtained (in milliseconds) for the test files. Comparing these times with those obtained in the previous exercise, conclude to what extent the desired objective was achieved (significantly reducing the execution time needed to obtain the solution).

Repetitions: 10000		
TestFile	Time(ms)	Bins
test00.txt	152	4
test01.txt	109	1
test02.txt	2011	8
test03.txt	127	2
test04.txt	526	10
test05.txt	1116	14
test06.txt	117	7
test07.txt	151	8
test08.txt	12262	16
test09.txt	21361	33

Compared to simple backtracking (without pruning), Branch and Bound significantly reduces execution time by pruning good solutions early and exploring better configurations before performing the actual algorithm.

I think I made a mistake in the previous exercise as I implemented a lot of pruning, which made the backtracking algorithm more efficient.

However, if we compare these execution times with the simple backtracking ones, we can clearly see a very significant performance improvement when trying to solve the same problem.